

VulTriNet: A software vulnerability detection method based on tri-channel network

Yiyao Yang, Youjian Yao, Xiao Lv, Wen Chen*

School of Cyber Science and Engineering, Sichuan University, Chengdu, 610041, Sichuan, China

ARTICLE INFO

Keywords:

Vulnerability detection
Software security
Code representation
Deep learning
RGB image

ABSTRACT

Context: Software vulnerabilities represent a critical concern in cybersecurity. As vulnerability patterns become increasingly complex, advanced detection methods are needed to fully analyze them. Recent studies have treated source codes as text using natural language processing (NLP) techniques. Subsequent advancements transformed programs into intermediate representations, utilizing graph neural network (GNN) for vulnerability learning. However, these approaches exhibit limitations in software vulnerability detection, as they fail to comprehensively analyze the features of source code.

Objective: To solve this problem, we proposed a novel vulnerability detection method based on a tri-channel network (VulTriNet), which enables comprehensive analysis of source code and effective vulnerability detection.

Methods: The Method integrates two graph-based and one textual code representation using three distinct methods to transform functions into multiple forms. Then, inspired by the RGB three-channel concept in the image domain, VulTriNet generates corresponding embedding vectors for these transformed representations, which are subsequently merged into a unified three-channel feature matrix. Finally, there is a CNN model integrated with attention mechanisms to improve the capability of detecting vulnerabilities.

Results: Experimental results demonstrated that, compared to five state-of-the-art approaches, VulTriNet achieves, on average across different datasets: a 4.89% improvement in accuracy, a 3.41% increase in TNR, a 4.09% gain in TPR, and a 4.18% boost in F1-score.

Conclusion: These results indicate that VulTriNet is more accurate and effective than previous studies. This hybrid analysis model strengthens vulnerability detection capabilities by simultaneously preserving contextual understanding of code and awareness of its structural relationships.

1. Introduction

The rapid expansion in the scale of software programs has significantly increased system complexity, posing serious challenges to software security. Specifically, software vulnerabilities are typically defined as flaws in the specification, development, or configuration of software that may cause it to perform actions that violate established security policies [1]. According to the “Open Source Security and Risk Analysis Report” published by Synopsys [2], 81% of the audited codebases contain at least one vulnerability. Therefore, it is crucial to develop methods capable of detecting software vulnerabilities both quickly and accurately.

Traditional program analysis methods for software vulnerability detection rely on expert-defined vulnerability rules and patterns [3,4], which require a high level of expertise. Furthermore, as the number and complexity of vulnerabilities increase, relying on manual detection

is not only resource-intensive but also fails to achieve the expected performance [5].

In recent years, researchers have proposed some automated code vulnerability detection models [6–8], among which the deep learning method has gradually attracted lots of attention. Deep learning models have superior learning abilities, which can learn the distribution of vulnerabilities from code samples and automatically perform vulnerability detection. For example, in Refs. [9,10], the source code is converted into token sequences or code graph structures, and vulnerability detection is performed through deep Bi-directional LSTM and graph neural networks.

The effectiveness of deep learning models have been consistently demonstrated in the field of vulnerability detection [11–15]. However, we cannot afford to ignore that to achieve satisfactory detection performance, firstly codes must be converted into feature vectors suitable for dealing before putting them into the deep learning models.

* Corresponding author.

E-mail address: wenchen@scu.edu.cn (W. Chen).

<https://doi.org/10.1016/j.infsof.2025.107893>

Received 14 May 2025; Received in revised form 31 August 2025; Accepted 1 September 2025

Available online 15 September 2025

0950-5849/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

A straightforward approach is to treat source code as a human language and apply natural language processing (NLP) methods to detect vulnerabilities within the code. For example, in VulDeePecker [11], SySeVR [14], and subsequent studies, in order to extract the features of vulnerabilities, the code is treated as text and sliced in a specific manner. Natural language processing techniques are then applied, followed by recurrent neural networks (RNNs) to learn contextual relationships for classification. However, source code is not simply equivalent to natural language. Such approaches fail to capture the logic and structural properties of code. Previous studies have shown that relying on contextual semantic representations is insufficient for accurately detecting vulnerabilities, especially in cases involving complex logic. On the other hand, some researchers proposed to convert the codes into graphical representations, allowing for graph-based analysis to detect vulnerabilities. Devign [15], BGNN4VD [10], and Reveal [16] adopt graph neural networks (GNNs) to learn the structural features of functions. They convert source code into graphs based on AST, CFG, DFG, or PDG, and then vectorize the functions by learning the relationships among graph nodes. Finally, a classifier is used to perform vulnerability detection. It should be noted that graph-based representation methods primarily focus on structural features of code, resulting in the loss of fundamental semantic information embedded in the code. For example, in overflow vulnerabilities, semantic elements such as numeric values, function expression, and variable identifiers play a critical role and should not be overlooked. In summary, using either textual features or graph representations alone may lose some crucial information for vulnerability classification, limiting the effectiveness of deep learning in vulnerability detection.

In this paper, we aim to enhance vulnerability detection by integrating multiple code representation methods to comprehensively extract vulnerability-related features. Our approach is inspired by the classical image content recognition procedure, which extracts content features through three channels for CNN neural network [17]. We proposed a new vulnerability detection method VulTriNet, which processes each input function into three different ways to serve as the three channels: AST representation, PDG representation, and CodeBERT [18] representation. Abstract Syntax Tree (AST) is a hierarchical structure that captures the syntactic organization of code, representing elements including functions, loops, and conditionals as nodes and edges. The Program Dependence Graph (PDG) combines control dependencies and data dependencies to model how code elements interact. A pre-trained transformer model (BERT-based) that converts code into contextualized embeddings by learning text semantic patterns from large code corpora. Combining these three representations, VulTriNet can incorporate structural precision, dependency tracking, and semantic understanding for robust vulnerability detection. Then, we treat the three types of function representations as three channels to generate an image. Finally, a customized CNN model is designed for vulnerability detection based on the abstracted images. Inspired by the CBAM method [19], we incorporate Efficient Channel Attention for Deep Convolutional Neural Networks (ECA-Net) [20] layers and Spatial Attention layers [21] into the CNN to focus on channels and regions, adapting to the features learned from the three forms.

The performance of VulTriNet is demonstrated through the evaluations based on the SARD [22] and FFMpeg+QEMU [15] vulnerability datasets, with performance metrics including accuracy, true positive rate (TPR), true negative rate (TNR), and F1-score being systematically measured. The experimental results indicated that our approach outperforms 5 newly proposed vulnerability detection approaches [11, 15, 23–25]. The results indicate that the VulTriNet exhibits stronger feature extraction capabilities through its three-channel design, which facilitates more effective model learning and vulnerability detection.

In summary, this paper makes the following contributions:

- A novel multi-dimensional code representation is proposed, transforming functions into AST, PDG, and CodeBERT representations.

- A new vulnerability detection model, VulTriNet, is introduced. It inputs the transformed function representations into the three RGB channels and employs a CNN model with attention mechanisms for vulnerability detection.
- Extensive experiments are carried out to test VulTriNet. The results show that VulTriNet outperforms all baseline approaches, demonstrating the effectiveness and soundness of the proposed design.

The rest of this paper is organized as follows. Section 2 discusses the preliminary. Section 3 introduces the proposed approach. Section 4 presents the experimental setup, while Section 5 demonstrates the effectiveness of the experiment. Section 6 discusses some important issues related to this paper. Section 7 reviews related work. Finally, Section 8 summarizes the study and outlines directions for future research.

2. Preliminary

It is well known that the use of neural networks requires converting source code into embedding vectors. The proposed method represents the source code in multiple forms and feeds these representations into a neural network.

2.1. The graphical representation of functions

Modern software analysis and transformation tasks often require structured representations of source code that go beyond raw text. Two prominent graph-based models are the **Abstract Syntax Tree (AST)** and the **Program Dependence Graph (PDG)**.

An AST is a hierarchical, tree-structured representation of the syntactic structure of source code. Given the importance of preserving hierarchical structure, the AST is transformed into a vector in a predefined order, ensuring that the positions of nodes within the hierarchy are appropriately reflected. In this way, a semantically meaningful sentence can be generated for each function.

In graphical representations, the Control Flow Graph (CFG) is a visual representation used to describe the control flow relationships between various basic blocks in a program. The Data Flow Graph (DFG) is a graph that represents the data dependencies within the program. A Program Dependence Graph (PDG) is a directed graph that models semantic dependencies between program statements. It combines control dependence and data dependence to represent how statements influence one another during execution. In this paper, we utilize Joern [26] and CodeSensor [27] to derive the PDG and the AST.

2.2. Representations from transformers

When it comes to embedding source code as vectors, it is straightforward to think of treating the source code as text and using methods like Word2Vec [28] or Sent2Vec [29] for embedding. However, the word vectors generated by these methods may lack adaptability to the specific structural and text semantic patterns of source code. To address these issues, the BERT model was proposed.

Built on the Transformer architecture [30], BERT leverages bidirectional context to generate contextualized embeddings for words or subword tokens. BERT dynamically adjusts word representations based on their surrounding context, enabling deeper semantic understanding. The structure of BERT is based on the Transformer encoder. BERT uses the Transformer architecture by employing self-attention to understand relationships between words in a sentence.

In [18], CodeBERT was introduced, a bimodal training model that works with both natural language (NL) and programming language (PL). It is developed based on the same model architecture as Roberta-Base [31], using a multi-layer bidirectional transformer as the model architecture for CodeBERT.

Specifically, during the pre-training phase, the researchers set the input as a combination of two segments with a special separator token, i.e., $[CLS], w_1, w_2, \dots, w_n, [SEP], c_1, c_2, \dots, c_m, [EOS]$. $[CLS]$ serves as a special beginning token, and its hidden representation is considered as the aggregated sequence representation for classification or ranking. The authors treat the natural language (NL) as a sequence of words, which are split into WordPieces, while a piece of code is treated as a token sequence. The paper used both parallel data of natural language-code pair, called bimodal data, and unpaired natural language text and code, called unimodal data, during training.

2.3. Convolutional Neural Networks and attention mechanisms

Usually, Attention mechanisms in CNN are techniques that allow models to focus on the most relevant parts of the input, mimicking human visual attention [32]. In the context of images, attention helps the model selectively emphasize important features while suppressing irrelevant ones. In CNNs, attention mechanisms are often integrated to enhance feature representation by assigning different importance weights to different regions of the input.

The two commonly used attention mechanisms in CNN are Channel Attention and Spatial Attention. Channel attention helps the model focus on important feature channels, allowing it to emphasize certain feature maps over others based on their relevance. Spatial attention focuses on specific regions or pixels in the image. It learns where to focus spatially, emphasizing regions that contribute most to the task at hand. Here, we focus on a type of channel attention mechanism known as SENet (Squeeze-and-Excitation Network) [33].

SENet consists of a series of SE blocks, and the processing of a single SE block is divided into two stages: **Squeeze** and **Excitation**. Specifically, let the input be $\mathbf{X}$. After a series of convolutional operations, we obtain the feature map $\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_c]$ where $\mathbf{U} \in \mathbb{R}^{H \times W \times C}$. The squeeze operation performs Global Average Pooling on the feature map to generate a compressed global feature vector, effectively addressing channel dependency utilization. Here, Z_c represents the c th element of $\mathbf{Z}$, and F_{sq} denotes the squeeze operation, which is computed as:

$$Z_c = F_{sq}(\mathbf{u}_c) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W u_c(i, j) \quad (1)$$

To leverage the information aggregated by Squeeze, the next step is Excitation. This step uses two fully connected layers to compute the weights for each channel in the feature map, capturing channel-wise dependencies. Here, σ represents the ReLU function, δ represents the Sigmoid function.

$$s = F_{ex}(z, W) = \sigma(g(z, W)) = \sigma(W_2 \delta(W_1 z)) \quad (2)$$

The final output of the block is derived by rescaling the transformation output $\mathbf{U}$ using the activation:

$$\tilde{\mathbf{x}}_c = \mathbf{F}_{scale}(\mathbf{u}_c, s_c) = s_c \cdot \mathbf{u}_c \quad (3)$$

The reweighted feature map is denoted as $\tilde{\mathbf{X}} = [\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \dots, \tilde{\mathbf{x}}_c]$, where $F_{scale}(\mathbf{u}_c, s_c)$ represents the channel-wise multiplication between the feature map $\mathbf{u}_c \in \mathbb{R}^{H \times W}$ and the scalar s_c . SENet can adaptively allocate channel-wise weights through this mechanism. In the subsequent sections, we will present an improved method based on this architecture.

3. Approach

This section presents **VulTriNet**, an efficient and novel model for source code vulnerability detection. As shown in Fig. 1, VulTriNet consists of three steps: function transformation, embedding vector generation, and classification. In **Step 1**, the function transformation explains how the function is transformed. Here, the code is cleaned up to produce a text representation, which is then further normalized to

generate the function's AST and PDG. In **Step 2**, embedding vector generation converts the function's representation into a vector. For the function's AST, we apply depth-first traversal (DFT) and use Word2Vec to generate the vector representation. The PDG is embedded using Sent2Vec, while the code text is processed through the CodeBERT model. In **Step 3**, we input the generated tri-channel image into the CNN for classification. The classification performs binary classification to determine whether the function contains any vulnerabilities.

3.1. Function transformation

VulTriNet is a function-level source code vulnerability detection method, aiming to obtain the desired function representation. To achieve this, some preprocessing of the source code is required. First, we need to remove all empty lines and spaces for formatting purposes and delete all useless comments. Next, to generate a structure-focused PDG and AST, we need to abstract and normalize the source code while preserving its semantics. Therefore, we perform the following operations [34]:

- (1) Map user-defined variable names to VAR1, VAR2, and so on.
- (2) Map user-defined function names to FUN1, FUN2, and so on.

An example of how to abstract and normalize a function is given in Fig. 2. Modifying custom function names and variable names allows the focus to remain on the structural aspects of the function without altering the program's semantics.

After preprocessing the source code, Joern [26], an open-source C/C++ source code analysis tool, it employed to generate the corresponding function's PDG. It is an open-source tool for static code analysis that can be used to understand the structure of the code. By utilizing Joern, as shown in Fig. 3, it gives an example of the PDG obtained for a specific function. In this graph, blue lines represent control flow, while red lines indicate data flow. It identifies data dependencies and control dependencies, and each element in the PDG corresponds to a line of code within the function.

An open-source code analysis tool, Codesensor [27], is also utilized to obtain the corresponding AST of the function in a serialized format. In the paper, the authors assume that vulnerable programming patterns are related to many security vulnerabilities. By analyzing the program's AST, these patterns can be revealed. The AST is a syntactical structure of the source code that describes the relationships between the components in a hierarchical tree view of the code, representing the function-level control flow. Compared to a Control Flow Graph (CFG), the AST provides a more natural representation of the program at the function level and retains more source code information. Codesensor is a robust parser that does not require complex dependencies based on the concept of island grammars [35]. Its input is simply the source code of a function, and its output is the function's AST.

3.2. The embedding vector generation

The purpose of this stage is to transform the function's representation into an embedding vector that can be used by a neural network. Specifically, the three different function representations are encoded into fixed-dimensional feature matrices. These matrices are then treated as the three channels of an RGB image and combined into a single 3D feature matrix. The following sections introduce each of the three encoding methods in detail.

(1) AST embedding: After the source code file is provided to CodeSensor, the corresponding parsed AST is automatically generated in a serialized format. Fig. 4 lists the structure of the tree in tabular form. For example, the first row of the table indicates that the function name is foo, and its return value as the root of the tree is int. The second row indicates that some parameters are received here, and it is the first child of the root node. However, even if there is a serialized AST of a function, it does not directly provide a suitable embedding

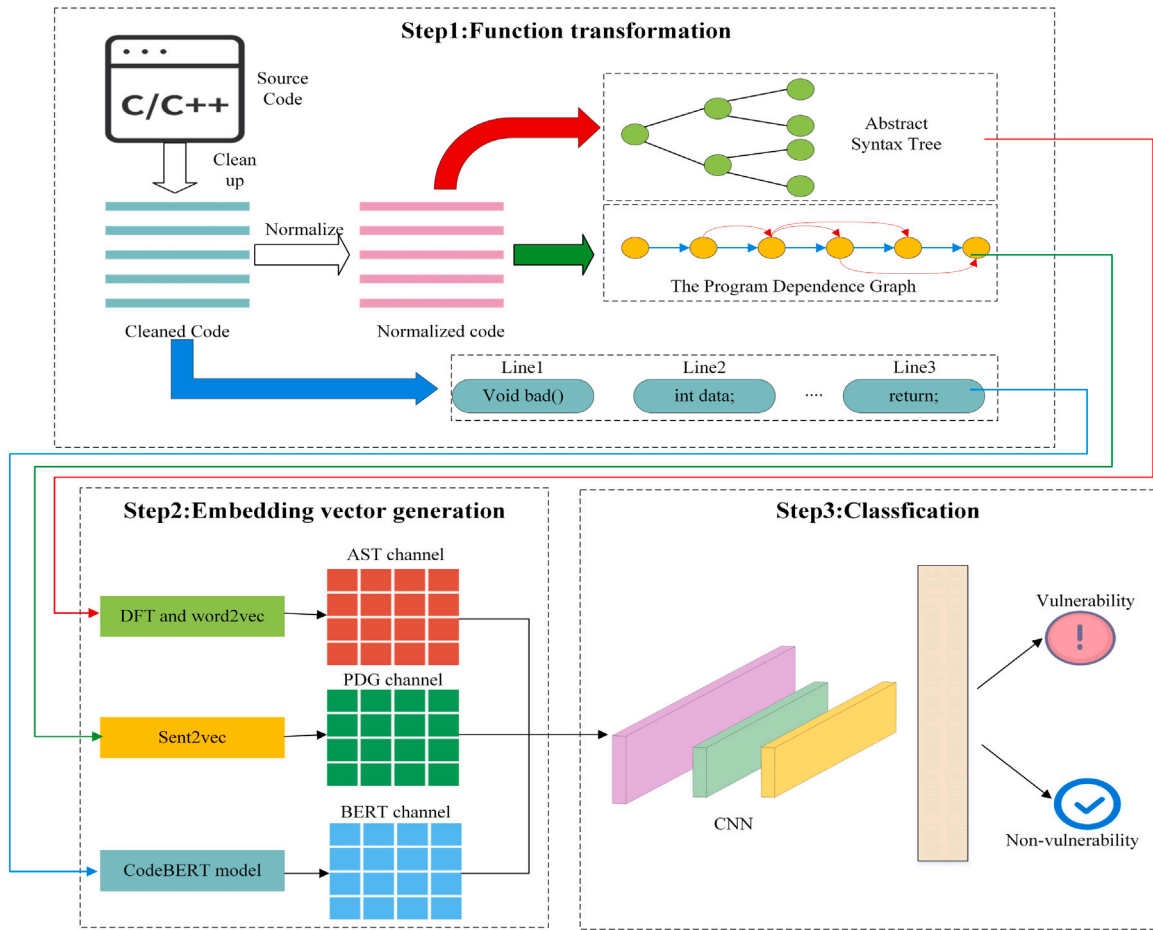


Fig. 1. Overview of VulTriNet.

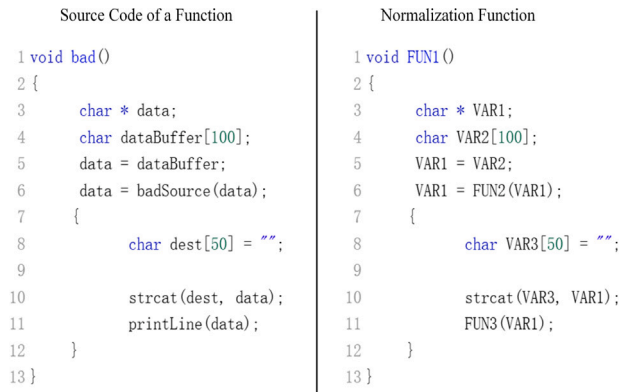


Fig. 2. Source code normalization.

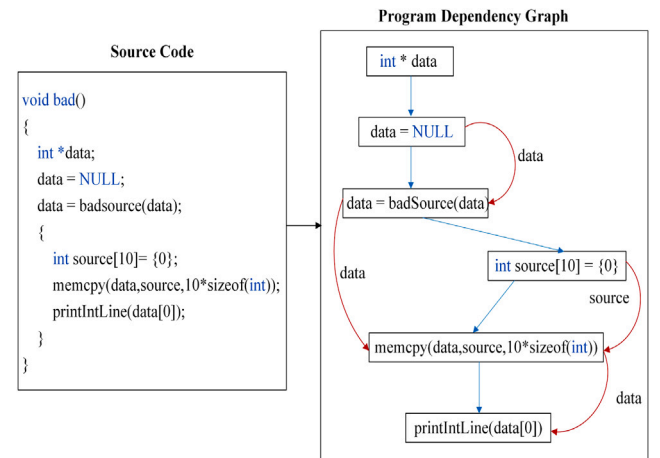


Fig. 3. The program dependency graph (PDG) example.

vector. Therefore, based on these serialized ASTs, it needs to traverse them in a uniform order to assemble their elements into a vector. Previous research has demonstrated that the order of elements in the vector is important, as their sequence partially reflects the hierarchical position of nodes in the AST. Inspired by Lin et al. [36], we adopt depth-first traversal (DFT) to map the AST elements to vectors in this paper. The mapping of the AST from the above example to the text vector format is: [foo, int, params, param, int, a, stmts, decl, int, c, op, =, decl, ...]. The first element of the vector represents the name of the root function, and the second element int indicates the return type of the function foo. The third element params, represents the

parameters the function accepts. The fourth and fifth elements are params and int, indicating that one parameter accepts elements of type int. For the elements of these vectors, which are arranged in order, we can treat them as “sentences” with semantic meaning. After ensuring the “sentences”, the code semantics still need to be captured through word embeddings. Traditional embedding methods like one-hot can convert words into vectors, but may overlook the potential relationships between words. Therefore, we apply Word2vec for word embedding and set the embedding dimension of Word2vec to 128.

		#Type	#Depth	Value 1	Value 2
1	int foo(int a){	func	0	int	foo
2	int c = 0;	params	1		
3	/*invoke function bar() */	param	2	int	a
4	int b = bar(a);	stmts	1		
5	if(a == 10){	decl	2	int	c
6	c = a + b;	op	2	=	
7	}	decl	2	int	b
8	return c;	op	2	=	
9	}	call	3	bar	
		arg	4	a	
		if	2	(a == 10)	
		cond	3	a == 10	
		op	4	==	
		stmts	3		
		op	4	=	
		op	4	+	
		return	2	c	

Fig. 4. An example of a C function and its serialized AST.

This gives us a fixed-dimensional feature matrix representation of the function.

(2) PDG embedding: After processing with Joern, a standardized PDG of the function is obtained. In the papers [25,37], the effectiveness of applying and how to apply PDG for vulnerability detection has been thoroughly demonstrated. Here, we adopt the methods from these two papers. In the generated PDG, each node represents a line of code, while the edges capture the control and data flow between them. Since the lines of code are regarded as sentences and are converted into fixed-dimensional vectors through embedding techniques. In particular, it has been demonstrated in the paper [37] that Sent2vec outperforms methods like Node2vec [38] in terms of graph node embedding. Therefore, we use the unsupervised Sent2vec to get the embedding vector, and based on their findings, the dimension is set to 128.

(3) CodeBERT embedding: To process source code text, CodeBERT is selected due to its demonstrated effectiveness in capturing code semantics. Compared to BERT, CodeBERT is specifically pre-trained on programming languages, enabling a better understanding of code syntax and semantics. In contrast to GraphCodeBERT, which emphasizes structural representations, CodeBERT focuses more on token-level learning, making it more suitable for semantic understanding of source code.

We utilize the C language pre-trained model provided in [39] for operations. An embedding vector representation of a function is obtained by leveraging the pre-trained CodeBERT model. After obtaining the CodeBERT embedding vector, a practical issue arises: The output of the BERT model is 768-dimensional. However, for compatibility in subsequent neural network processing, it is necessary for all three embeddings to have the same dimension. Setting the other two embeddings to 768 dimensions would be too large, and due to the size of our dataset, 768-dimensional vectors would impose a significant load on our hardware. Therefore, it is necessary to reduce the 768-dimensional vector to 128 dimensions.

Inspired by the BERT-Whitening in [40], we address the problem of obtaining sentence embeddings from CodeBERT by leveraging a method that enhances semantic similarity between sentences while reducing dimensionality to accelerate processing. BERT-Whitening uses a simple whitening operation to transform the embedding vectors into an isotropic space. There is a set of sentence vectors, which can also be written as row vectors $\{\tilde{x}_i\}_{i=1}^N$. Then, $\{\tilde{x}_i\}_{i=1}^N$ undergoes the following linear transformation:

$$\tilde{x}_i = (x_i - \mu)W \quad (4)$$

The above formula corresponds to the whitening operation in machine learning. Next, it is necessary to ensure that the mean value is

zero:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (5)$$

Next, solve for W , and let the covariance of the original data be denoted as:

$$\Sigma = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^T (x_i - \mu) \quad (6)$$

The transformed covariance matrix is $\tilde{\Sigma} = W^T \Sigma W$. Then, to ensure that the transformed covariance matrix becomes the identity matrix, the following equation must be solved.

$$W^T \Sigma W = I \quad (7)$$

Then, a simple transformation is applied to it:

$$\begin{aligned} \Sigma &= (W^T)^{-1} W^{-1} \\ &= (W^{-1})^T W^{-1} \end{aligned} \quad (8)$$

Since the covariance matrix Σ is a positive definite symmetric matrix, it satisfies the singular value decomposition (SVD) [41], which is given by:

$$\Sigma = U \Lambda U^T \quad (9)$$

Here, U is an orthogonal matrix, while Λ is a diagonal matrix. Therefore, let $W^{-1} = \sqrt{\Lambda} U^T$, which leads to the final result:

$$W = U \sqrt{\Lambda^{-1}} \quad (10)$$

Through the above operations, it is evident that the original covariance matrix of the sentence embeddings can be transformed into an identity matrix. Since each diagonal element of the diagonal matrix Λ measures the variance of the corresponding one-dimensional data, a small value indicates that the variation of that dimension's feature is also small and negligible. Therefore, the corresponding sentence vector can be embedded into a lower-dimensional space. In the previous SVD decomposition, the elements in the diagonal matrix Λ are already sorted in descending order. Thus, theoretically, similar to PCA (Principal Component Analysis) [42], we only need to retain the first k columns of W to achieve this dimensionality reduction effect. Here, we choose $k=128$.

3.3. Classification

In the previous stage, a three-dimensional feature matrix was obtained, serving as the image representation of the function. We use Convolutional Neural Networks (CNN) in deep learning for this purpose, as CNNs have demonstrated excellent performance. This paper employs the previously processed embedding vectors to replace the original RGB channels in the image with AST channels, PDG channels, and CodeBERT channels. The proposed CNN model is built upon TextCNN [43], which is a CNN model suitable for text processing. An attention mechanism is introduced by incorporating ECA-Net and spatial attention mechanisms [21], aiming to better select the weights among the three channels and enhance feature interaction.

The structure of the neural network classification model is illustrated in Fig. 5. Our model combines the three types of vectors into a three-channel feature matrix and processes it as an image. Following this, our model employs three attention-based convolutional blocks, each integrating a standard convolutional layer (ReLU-activated), efficient channel attention (ECA), spatial attention mechanisms (SAM), and selective max-pooling for spatial downsampling, progressively expanding channel depth across the blocks. Next, the model includes an adaptive average pooling layer that reduces the spatial dimensions to a single feature vector and incorporates dropout at this stage to prevent overfitting. Finally, there is a fully connected layer that outputs a two-class probability distribution, indicating the likelihood of the input

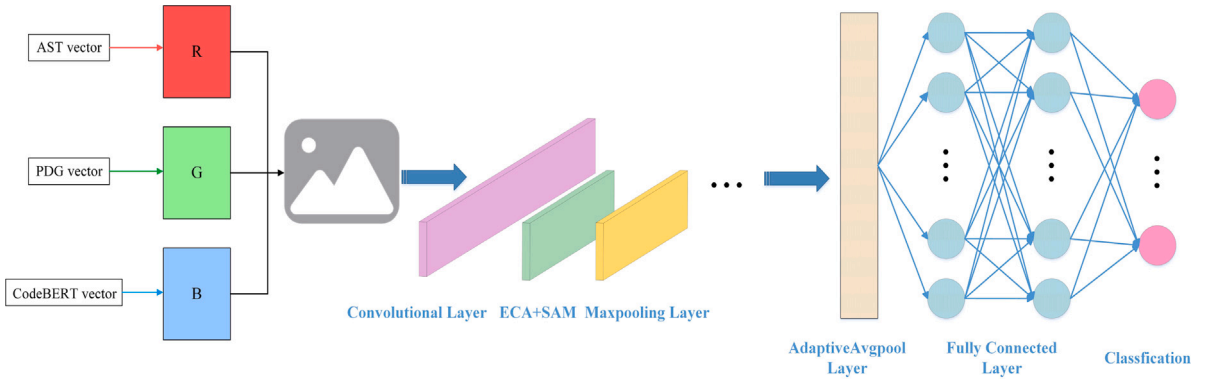


Fig. 5. The structure of CNN classification.

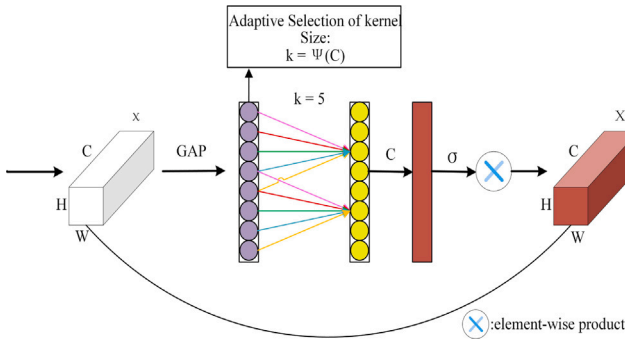


Fig. 6. The efficient channel attention(ECA) module.

code being vulnerable or not. This architecture leverages attention mechanisms to focus on salient features, ensuring robust and efficient vulnerability detection.

Fig. 6 is an overview of the efficient channel attention(ECA) module. ECA first obtains aggregated features through Global Average Pooling (GAP) and then performs a fast 1D convolution of size k to generate channel weights, where k is adaptively determined based on channel dimension C . Compared to the previously introduced SENet, ECA not only establishes relationships between all channels but also avoids dimensionality reduction while ensuring computational efficiency. Specifically, ECA removes the dimensionality reduction operation and directly models the original channel dimension. It uses 1D convolution instead of fully connected layers, capturing only the interactions between locally adjacent channels, which reduces complexity. The modifications made are as follows:

$$s = \sigma(\text{Conv1D}_k(z)) \quad (11)$$

Here, z is the same as in Section 2.3, representing the result after the GAP operation. Conv1D denotes a 1D convolution with a kernel size of k , and σ is a Sigmoid function. The size k is dynamically determined based on the channel dimension C as follows:

$$k = \psi(C) = \left\lceil \frac{\log_2(C)}{\gamma} + \frac{b}{\gamma} \right\rceil_{\text{odd}} \quad (12)$$

where γ is set to 2 and b is set to 1. The odd ensures that k is the nearest odd number. After channel attention processing, SAM further focuses on the importance of each spatial position. It first performs average pooling and max pooling on the feature map along the channel dimension, generating two 2D feature maps. Then, these two feature maps are concatenated and passed through a convolutional layer (using a 7×7 kernel) to produce a spatial attention map. Finally, this attention map is multiplied by the feature map to achieve feature recalibration in the spatial dimension.

Table 1

The statistics of datasets.

Dataset	Vul	No-Vul	Total	Ratio
Sard	13,942	27,107	41,049	1:1.94
FFmpeg+QEMU	10,004	12,874	22,878	1:1.28

In addition, CNN requires matrices of the same size, but the source code has different numbers of lines. We performed a simple statistical analysis on the line counts in the dataset, and experimental results indicate that 225 lines is a reasonable number. For code exceeding 225 lines, content after the 225th line is truncated, while code with fewer than 225 lines is padded with zeros. The generated image size is $3 \times 225 \times 128$, where 3 represents the number of channels, 225 is the number of rows in the feature matrix, and 128 corresponds to the number of columns in the feature matrix.

4. Experiments setup

4.1. Datasets

In our work, we focus on function-level detection of C/C++ vulnerabilities and conduct experiments on two well-known datasets. The first dataset, **SARD** (The Software Assurance Reference Dataset) [22], is a publicly available resource curated by the National Institute of Standards and Technology (NIST) [44] to support research and development in software security analysis. It provides a collection of code samples, including intentionally vulnerable and patched (non-vulnerable) code snippets, designed to benchmark and improve static analysis tools and machine learning-based detection systems. The dataset covers various Common Weakness Enumeration (CWE) categories, such as buffer errors (CWE-119), offering granular code gadgets or function-level samples to support fine-grained vulnerability analysis. The other dataset used is the **FFmpeg+QEMU** vulnerability dataset, provided by Zhou et al. [15]. As the SARD dataset is artificially constructed, the FFmpeg+QEMU dataset, built from real-world projects, is additionally selected to validate the effectiveness of the proposed method. Furthermore, compared to other datasets, FFmpeg+QEMU exhibits better class balance, contributing to its widespread adoption in prior research. The final statistics of the two datasets are presented in Table 1.

4.2. Evaluation metrics

We use the following widely used evaluation metrics to measure the model's performance:

TPR (True Positive Rate): The proportion of actual positive samples that are correctly predicted as positive.

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (13)$$

TNR (True Negative Rate): The proportion of actual negative samples that are correctly predicted as negative.

$$TNR = \frac{TN}{FP + TN} \quad (14)$$

The F1-score is the harmonic mean of precision and recall, used to evaluate the performance of a classification model.

$$F1\text{-score} = \frac{2 \cdot TP}{2 \cdot TP + FN + FP} \quad (15)$$

ACC(Accuracy) is the ratio of correctly predicted instances to the total instances in a dataset, used to measure the overall performance of a classification model.

$$ACC = \frac{TP + FN}{TP + TN + FN + FP} \quad (16)$$

4.3. Baselines

Several state-of-the-art approaches are selected as baseline comparisons to demonstrate the effectiveness of the VulTriNet method. All baseline models are configured using hyperparameters strictly adhering to the specifications outlined in their respective original publications to ensure experimental reproducibility and fairness in comparative analysis.

The Design [15] method is a machine learning-based approach for automated vulnerability detection in source code. It employs Graph Neural Networks (GNNs) to analyze both semantic and structural patterns by constructing a Code Property Graph (CPG). The method classifies code snippets as vulnerable or non-vulnerable through a graph-level binary classification mechanism.

VulDeepeer [11] is a deep learning-based vulnerability detection method that treats source code as natural language, using bidirectional Long Short-Term Memory (Bi-LSTM) networks to analyze semantic and syntactic patterns in code. It first converts code into code gadgets and encodes them into vectors via word embeddings trained on code-specific tokens. The Bi-LSTM model processes these embeddings to classify snippets as vulnerable or non-vulnerable.

LineVul [23] is a deep learning-based vulnerability detection method designed to identify security flaws in source code by leveraging transformer models and line-level analysis. It processes code as a sequence of tokens, utilizing a pre-trained transformer architecture to capture contextual relationships and semantic patterns across individual code lines.

GraphCodeBERT [24] is a pre-trained deep learning model designed for code representation by integrating semantic-level structural information through data flow graphs (DFGs). Built on the Transformer architecture, it employs a graph-guided masked attention mechanism to restrict attention to relevant nodes and edges in DFGs.

VulCNN [25] is a CNN-based method that normalizes the function source code to obtain its corresponding PDG. After embedding with sent2vec, it uses the concept of social network centrality to multiply and transform the embedding vector into a three channels image and finally uses TextCNN for classification.

4.4. Details

The paper experiments on a machine with an Intel Xeon Silver 4210R CPU @ 2.40 GHz, 128 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU. In our development environment, we used Python 3.8.5 and Joern version 1.1.1100 to generate PDGs. We employed PyTorch 1.10.1 and downloaded the BERT model from Hugging Face. During training, we utilized Cross Entropy as our loss function and Adam [45] as our optimizer. The batch size was set to 32, with an initial learning rate of 0.001.

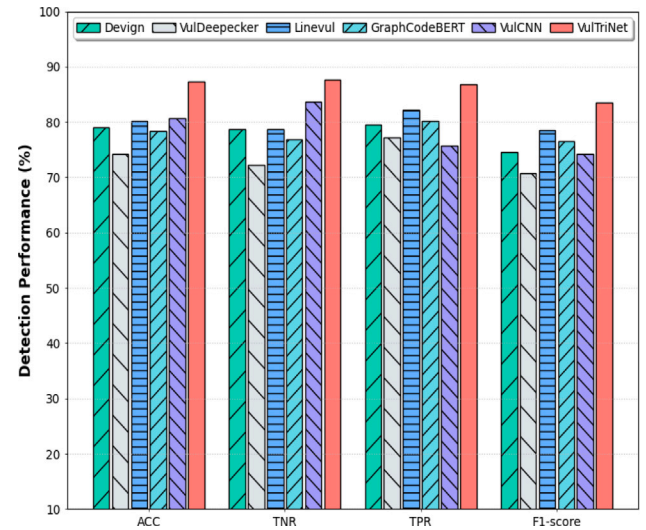


Fig. 7. The performance of VulTriNet on SARD.

4.5. Research Questions

We evaluate the performance of VulTriNet on several datasets, aiming to answer the following Research Questions (RQs):

- **RQ1:** How does VulTriNet perform in vulnerability detection compared to several state-of-the-art methods across different datasets?
- **RQ2:** Are the components and design choices in VulTriNet effective, and how do they compare with other commonly used designs?
- **RQ3:** What is the impact of several key parameters on the performance of our method?
- **RQ4:** Are the statistical results of VulTriNet stable?

5. Experimental results

5.1. RQ1: Effectiveness of VulTriNet

This section compares the proposed approach with five state-of-the-art vulnerability detection methods, with evaluation conducted using TPR, TNR, accuracy, and F1-score.

As shown in Fig. 7, our approach outperforms all baselines on the artificially constructed and relatively balanced SARD dataset across all evaluation metrics. Specifically, our approach achieved an Accuracy of 87.32%, a TNR of 87.63%, a TPR of 86.78%, and an F1-score of 83.47%. Compared to the best-performing baseline approach, we achieved improvements of 6.64%, 4.01%, 4.64%, and 4.88% in these four evaluation metrics, respectively. Design is a classic graph neural network-based method that inspired our work. While it represents a function as a joint graph, the direct conversion into a graph may lead to the loss of some textual semantic information from the original function. Additionally, its use of a simple Word2Vec embedding may introduce certain limitations. VulDeepeer utilizes code slicing techniques to generate Code Gadgets for classification. This slicing-based approach enables vulnerable samples to be represented in a finer-grained manner, leading to strong performance in classifying vulnerable samples and achieving a high TPR. However, this approach does not achieve satisfactory results in this context, as it relies solely on data-flow information. Moreover, treating sliced code purely as text may lead to the loss of important structural and semantic information. GraphCodeBERT and LineVul are both models based on

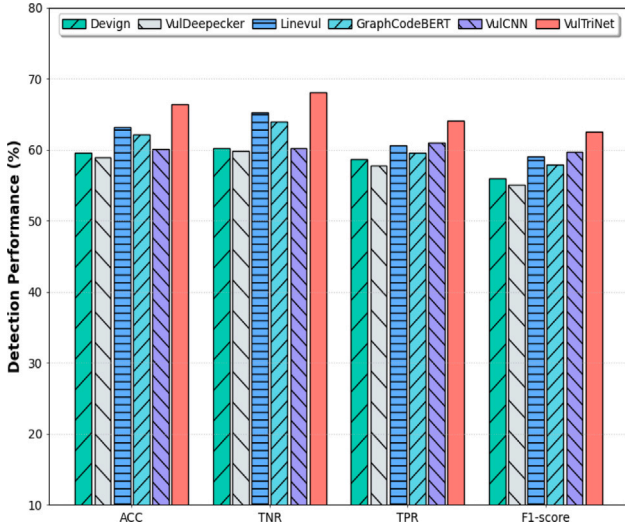


Fig. 8. The performance of VulTriNet on FFmpeg+QEMU.

pre-training. GraphCodeBERT is the first pre-trained model that leverages the semantic structure of code to learn code representations, and it is based on DFG (Data Flow Graph). However, while the data flow-based approach may perform well in some tasks, it is insufficient for fully representing code in the domain of vulnerability detection. LineVul first applies BPE subword tokenization and then, similar to our approach, utilizes the CodeBERT model, followed by a simple linear layer for classification. While this inspired our use of the CodeBERT pre-trained model, experimental results indicate that merely leveraging CodeBERT does not yield optimal classification performance. LineVul relies solely on CodeBERT to learn the semantic information of the code while losing structural information. This demonstrates that simply incorporating pre-trained models in vulnerability detection does not necessarily lead to satisfactory results. Finally, our approach builds upon and improves VulCNN. Unlike VulCNN, which solely relies on PDG and uses centrality to populate its three channels, our approach expands upon this foundation. Experimental results demonstrate that our approach achieves better performance by integrating and enhancing VulCNN's use of PDG. Compared to VulCNN, our approach more effectively captures both the syntactic information through AST and the semantic information through CodeBERT, leading to a significant improvement in performance.

In addition to conducting experiments on the synthetic SARD dataset, we also evaluated our approach on the more realistic FFmpeg+QEMU dataset. As shown in Fig. 8, the experimental results demonstrate that VulTriNet consistently achieves the best performance across different datasets. Compared to the best-performing alternative method, VulTriNet achieved improvements of 3.13% in ACC, 2.80% in TNR, 3.54% in TPR, and 3.48% in F1-score. Due to the unique distribution of the FFmpeg+QEMU dataset, all methods experienced a decline across the four evaluation metrics compared to the SARD dataset. However, VulTriNet still outperformed all other methods on every evaluation metric, demonstrating its superior ability to learn vulnerability features and perform accurate classification. In particular, although VulCNN performs well on the SARD dataset, its performance is relatively limited on the FFmpeg+QEMU dataset.

In conclusion, VulTriNet extends the feature representation and consistently achieves strong performance across different datasets, demonstrating its effectiveness and generalizability.

Table 2

The performance of VulTriNet's variants.

Approach	ACC	TNR	TPR	F1-score
VulTriNet-NA	83.84%	85.67%	80.74%	78.66%
VulTriNet-NP	84.64%	85.82%	82.64%	79.88%
VulTriNet-NB	78.35%	87.41%	62.84%	68.16%
VulTriNet-OA	71.95%	75.53%	69.87%	66.51%
VulTriNet-OP	73.19%	73.70%	72.34%	66.56%
VulTriNet-OB	76.03%	77.64%	73.28%	69.28%
VulTriNet-W	82.66%	82.37%	83.15%	77.96%
VulTriNet-S	84.36%	85.57%	82.00%	79.46%
VulTriNet-TC	82.10%	85.12%	76.96%	76.03%
VulTriNet-WA	84.42%	85.59%	82.43%	79.60%
VulTriNet-SE	85.98%	86.37%	85.33%	81.79%
VulTriNet	87.32%	87.63%	86.78%	83.47%

5.2. RQ2: Ablation study

This section presents several ablation experiments conducted to evaluate the proposed approach.

Effectiveness of Different Feature Channels. Our method utilizes multiple feature channels. To investigate their impact on the results, we conduct a comprehensive comparison between VulTriNet and its channel-specific variants, which are defined as follows:

VulTriNet-NA(No AST channel): VulTriNet without the AST channel, retaining only the PDG and BERT channels.

VulTriNet-NP(No PDG channel): VulTriNet without the PDG channel, retaining only the AST and BERT channels.

VulTriNet-NB(No BERT channel): VulTriNet without the BERT channel, retaining only the AST and PDG channels.

VulTriNet-OA(Only AST channel): VulTriNet discards the other two channels and only has the AST channel.

VulTriNet-OP(Only PDG channel): VulTriNet discards the other two channels and only has the PDG channel.

VulTriNet-OB(Only BERT channel): VulTriNet discards the other two channels and only has the BERT channel.

As shown in Table 2, the experimental results demonstrate that the best performance is achieved only when all three channels are present simultaneously. Specifically, the performance of the three single-channel variants is poor, being weaker in every metric compared to the two-channel variants that incorporate additional features. This indicates that the vulnerability features learned by a single channel are limited and demonstrates the effectiveness of multi-channel feature learning. Furthermore, the experimental results later confirm that the performance of the two-channel variants is inferior to that of the three-channel variant. The original VulTriNet outperforms its variants across all four evaluation metrics, achieving improvements of 2.68% in ACC, 0.22% in TNR, 4.14% in TPR, and 3.59% in F1-score compared to the best-performing variant, proving that each of the three channels plays an indispensable role in vulnerability detection. The absence of any one channel negatively impacts classification results. More specifically, the experimental results indicate that VulTriNet shows a significant difference in TPR and F1-score compared to its variants, proving that the three-channel architecture better captures vulnerability-related features and improves the detection of vulnerabilities, which further benefits our task. Notably, the performance drops significantly when the BERT channel is removed, indicating that the CodeBERT pre-trained model captures sufficient semantic knowledge, which is crucial for classification.

Effectiveness of CodeBERT. To investigate the impact of our CodeBERT embedding, we compare VulTriNet with two common embedding methods: Word2Vec, which provides vector representations of words, and Sent2Vec, which generates vector representations for entire sentences:

VulTriNet-W(Word2vec): VulTriNet replaces the CodeBERT channel with Word2Vec embeddings.

VulTriNet-S(Word2vec): VulTriNet replaces the CodeBERT channel with Sent2Vec embeddings.

As shown in Table 2, the experimental results demonstrate that our approach achieves the best performance across all four evaluation metrics, with improvements of 2.96% in ACC, 1.88% in TNR, 3.65% in TPR, and 4.01% in F1-score. Although Word2Vec captures word similarities and semantic relationships, it is inherently limited in directly handling phrases or sentences, which is critical for code representation at the line level. Furthermore, due to vector length constraints, the long embeddings generated by Word2Vec may suffer from information loss. While Sent2Vec performs better than Word2Vec by producing sentence-level embeddings, it still falls short of our approach. This is primarily because Sent2Vec is a text-based embedding model that does not account for the unique structural and semantic characteristics of code, making it less capable of differentiating logical variations. In contrast, our approach leverages CodeBERT embeddings, which are specifically designed for code representation. CodeBERT is pre-trained on a large corpus of code and comments using a transformer-based architecture that captures both syntactic structure and deep semantics, with an optimized tokenizer for handling code-specific symbols. Consequently, our approach not only addresses the limitations observed in both Word2Vec and Sent2Vec but also delivers superior classification performance.

Effectiveness of our CNN model and attention mechanism. To investigate the effectiveness of our CNN network, we employ other neural networks for the classification task.

VulTriNet-TC(TextCNN): VulTriNet employs TextCNN for the classification task.

VulTriNet-WA(Without Attention): VulTriNet without the attention mechanism module.

VulTriNet-SE(SENet): VulTriNet utilizes the SENet attention mechanism.

This experiment incrementally disassembles our classification model for validation. As shown in Table 2, all four evaluation metrics consistently improve as our model becomes progressively refined. TextCNN is a text classification model based on convolutional neural networks. It captures local semantic features in text using multiple one-dimensional convolutional kernels of different sizes. Its structure is simple, and it has fewer parameters. However, since it only applies a single-layer convolution, it fails to capture deeper global semantics. Compared to VulTriNet-TC, VulTriNet-WA, which utilizes our neural network architecture, achieves improvements of 2.32%, 0.47%, 5.47%, and 3.57% in ACC, TNR, TPR, and F1-score, respectively. Next, from the experimental results, it is evident that both VulTriNet-SE and VulTriNet outperform VulTriNet-WA, which does not use an attention mechanism, across all evaluation metrics. This demonstrates that the attention mechanism plays a crucial role in our classification model. Incorporating attention mechanisms enhances the capability of our CNN model in vulnerability detection. Among the methods that incorporate attention mechanisms, VulTriNet outperforms VulTriNet-SE, achieving improvements of 1.34%, 1.26%, 1.45%, and 1.68% in ACC, TNR, TPR, and F1-score, respectively. This indicates that our improved attention mechanism effectively enhances the model's focus on vulnerability-related features, further benefiting our task.

In conclusion, the methods incorporated in VulTriNet are all beneficial, collectively enhancing the effectiveness of our approach.

5.3. RQ3: Sensitivity to parameter

In this section, we experimentally verify the impact of key parameters on the results.

In the proposed approach, a CNN is employed as the classifier to determine the presence of vulnerabilities. Prior to classification, it is

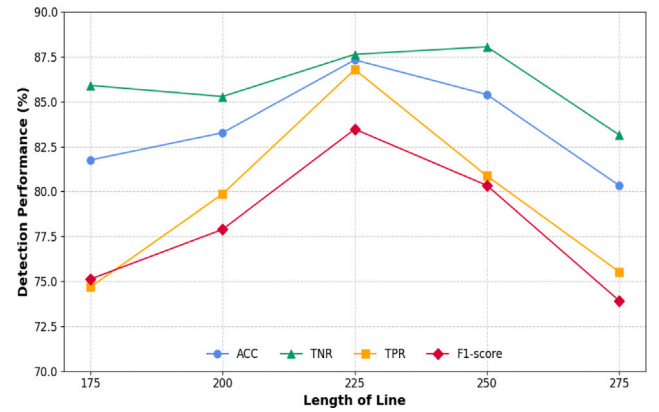


Fig. 9. Performance of the length of line.

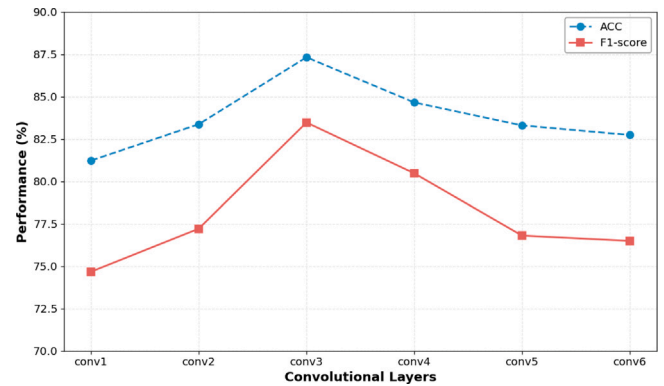


Fig. 10. Performance of the number of convolutional layers.

essential to preprocess the input data to ensure uniform formatting. Obviously, different samples have varying lines of code, resulting in significant length differences. Meanwhile, we use three channels as input, but their lengths are inconsistent, posing a new challenge. Therefore, it is crucial to accurately determine the length of the lines.

As shown in Fig. 9, when the length of the line is 225, it exhibits the best performance, achieving ACC, TNR, and TPR of 87.32%, 87.63%, and 86.78%, respectively. In terms of ACC, when the length of the line is 225, it outperforms other parameter choices, indicating that this parameter selection achieves better overall classification performance and more precise classification. Similarly, for TPR, the performance at a length of 225 significantly surpasses other choices, proving that this parameter selection is more effective in identifying vulnerable samples, aligning better with our objective. For TNR, the performance at a length of 225 is slightly lower than at 250, possibly because increasing the length within a certain range introduces additional padding, causing the classifier to lean toward classifying them as non-vulnerable samples. Furthermore, as verified in Fig. 9, the F1-score demonstrates that a length of 225 achieves far superior results compared to other parameter choices. In conclusion, we select 225 as the optimal length of the line.

Our classification model is based on CNN, where the number of convolutional layers is a key parameter. We designed experiments to study the impact of different numbers of convolutional layers on performance. “Conv1” represents using a single convolutional layer, “Conv2” represents using two convolutional layers, and so on. As shown in Fig. 10, using three convolutional layers achieves the best performance, showing a clear advantage over other configurations. When the number of convolutional layers is fewer than three, the model may fail to extract complex features effectively due to insufficient network depth, resulting in suboptimal performance. However, when the number of

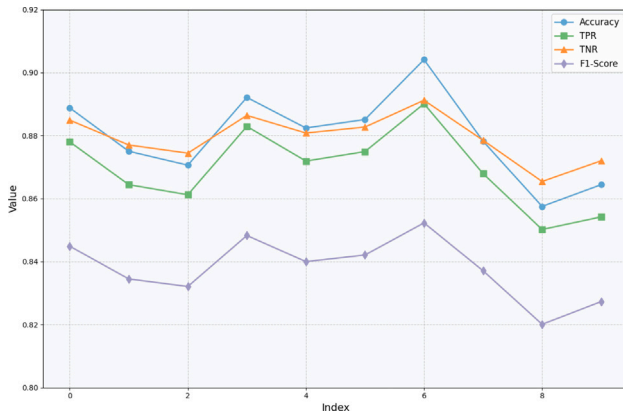


Fig. 11. Statistical analysis of experimental results.

convolutional layers exceeds three, the experimental results do not improve as expected. As demonstrated in [46], increasing the network depth beyond a certain threshold can lead to performance degradation. This indicates that simply stacking more layers does not necessarily yield better results. Based on experimental findings, three convolutional layers are selected for the task.

In conclusion, the experimental results demonstrate that setting the embedding vector length to 225 and employing a CNN architecture with three convolutional layers yields the best performance.

5.4. RQ4: Analysis of VulTriNet's stability

In this section, the stability of the VulTriNet is evaluated by conducting ten experiments under same experimental settings, with only the random seed being varied. The results are recorded for analysis.

As shown in Fig. 11, the results of ten independent experiments across four evaluation metrics are presented. While slight fluctuations are observed, they remain limited, indicating that the experimental outcomes are trustworthy. Specifically, the standard deviations of ACC, TPR, TNR, and F1-score are 0.013, 0.011, 0.007, and 0.009, respectively. Moreover, the range between the maximum and minimum values for each metric does not exceed 7%. These findings confirm that VulTriNet consistently maintains stable performance across multiple runs.

In conclusion, VulTriNet maintains a stable and trustworthy performance even in the presence of random variations, confirming the robustness of the proposed approach.

6. Discussions

6.1. Threats to validity

Experimental studies may encounter various threats to validity, such as internal threats, which concern the design and execution of the experiment, and external threats, which pertain to the generalizability and applicability of the results.

A threat to internal validity lies in the selection of experimental parameters, as these can significantly influence the outcomes. To mitigate this threat, we designed multiple experiments based on our dataset to evaluate the impact of various parameter settings.

Another potential threat to internal validity is related to our baseline experiments. To address this, we carefully reproduced the existing

implementations for each baseline. The baseline tools used in our experiments are publicly available replication packages, which we downloaded from GitHub. For each baseline, we strictly followed the instructions provided in the README files and used all default settings and parameters as specified. For LineVul and GraphCodeBERT, we downloaded the pretrained models from <https://huggingface.co/>, and for GraphCodeBERT, we additionally built a simple classifier to complete the evaluation. However, some approaches are not open-source, and our experiments rely on publicly available re-implementations provided by third parties on GitHub. Although these implementations have been widely adopted in the community, there may still be potential discrepancies or limitations compared to the original versions.

A threat to external validity is related to the programming languages used in our study. All the programs we analyzed were written in C/C++, so the effectiveness of our method may not be guaranteed on other programming languages. Nevertheless, our approach is theoretically generalizable and can potentially be applied to other commonly used programming languages, which we plan to explore in future work.

Another threat to external validity lies in the use of tools. We selected several tools to support our implementation, and these tools may have certain limitations. To mitigate this threat, we used the same versions of the tools as those adopted in the original methods wherever applicable, and we discarded samples that failed to be parsed, ensuring that all operations were performed only on successfully parsed data.

6.2. Discussion

6.2.1. Verification on different vulnerability types

To further demonstrate the effectiveness of VulTriNet, we selected the most representative and related baselines for a more detailed comparative experiment. We merged samples from our dataset that contained explicit CWE marking and supplemented them with additional entries from the NVD. The combined dataset was then partitioned according to CWE types. CWE types with too few samples were excluded from the analysis to ensure the statistical reliability of the experimental results. For the remaining types, stratified sampling was performed to proportionally select samples from each CWE category for training and testing VulTriNet and each baseline.

Fig. 12 presents the performance of VulTriNet and baselines across nine different CWE vulnerability types. As shown in the figure, VulTriNet achieves the best performance in nearly all evaluation metrics across most CWE categories. Specifically, for certain vulnerabilities such as CWE-126, all approaches perform generally, while for others like CWE-78, the performance is consistently strong. It is worth noting that since our dataset is primarily derived from synthetic sources like SARD, the results may be significantly influenced by the dataset itself and may not fully reflect real-world detection effectiveness. Nevertheless, the results indicate that VulTriNet exhibits stable and robust performance under various conditions. Although VulTriNet may not achieve the best score on every metric for every CWE type, it consistently maintains strong performance without significant drops in specific vulnerability types, unlike some baselines.

This result further demonstrates that VulTriNet possesses a comprehensive feature extraction capability that effectively supports vulnerability classification. Here, we give a simple example to illustrate this point more specifically. As shown in Fig. 13, this is a simple example of **CWE-369: Divide By Zero**, where the vulnerability arises from the lack of validation on the data variable before the final division operation. Our approach can precisely trace the PDG and infer whether the division is safe by analyzing the data and control flows originating from the **fscanf** input, which was not validated.

However, classification based on PDG tracking does not always yield ideal results. Our research focuses on vulnerability detection at the

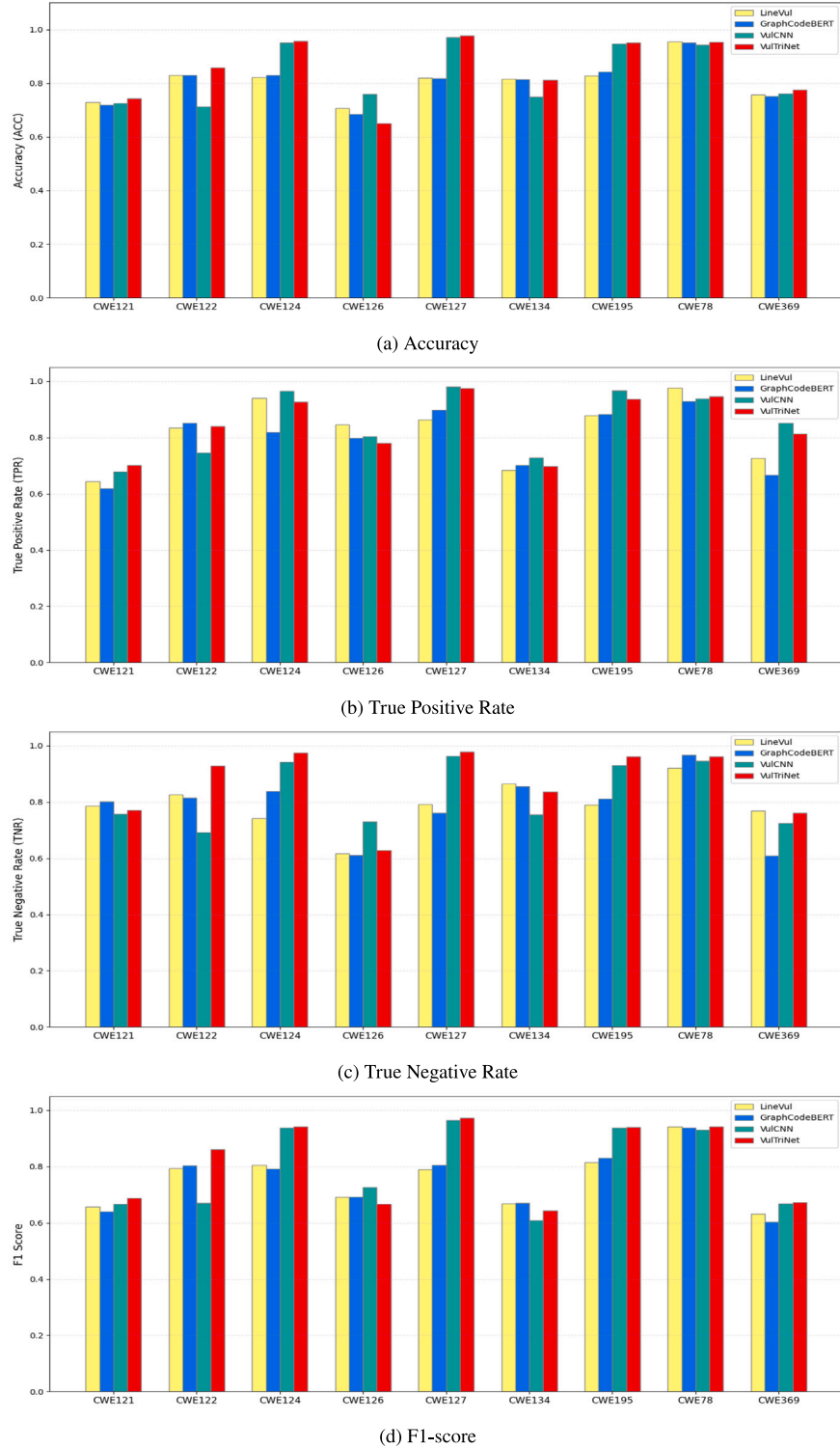


Fig. 12. Performance of VulTriNet and baselines on different CWE types.

function-level, and there will be some issues, as shown in Fig. 14. As observed in the figure, PDG-based tracking becomes ineffective, as it is difficult to determine the presence of vulnerabilities through control and data flow when external function calls are involved. Nevertheless, in many cases where external calls hinder vulnerability detection, our CodeBERT-based semantic channel helps mitigate this limitation.

By leveraging text semantic information, we can capture statistical patterns. For instance, in Fig. 14, the function name exhibits distinctive characteristics and is repeatedly invoked in other functions. This allows us to statistically infer whether it is a potentially dangerous function. Additionally, constants like “100” mentioned earlier might serve as crucial indicators — text semantic models such as Sent2Vec

```

1 void CWE369_bad()
2 {
3     int data;
4     int *dataPtr1 = &data;
5     int *dataPtr2 = &data;
6     data = -1;
7     {
8         int data = *dataPtr1;
9         fscanf(stdin, "%d", &data);
10        *dataPtr1 = data;
11    }
12    {
13        int data = *dataPtr2;
14        printIntLine(100 % data);
15    }
16 }

```

Fig. 13. A CWE-369 function example.

```

1 Void CWE121_bad()
2 {
3     char * data;
4     char dataBuffer[100];
5     data = dataBuffer;
6     memset(data, 'A', 100-1);
7     data[100-1] = '\0';
8
9     CWE121_Stack_Based_Buffer_Overflow__CWE806_c
10    har_declare_loop_53b_Sink(data);
11 }

```

Fig. 14. A CWE-121 function example.

or BERT are well-suited to capture such contextual clues. In real-world scenarios, variable and function names may themselves carry vulnerability-relevant information. Therefore, the learning of semantic information constitutes a crucial component of vulnerability detection and should not be overlooked.

6.2.2. Detection efficiency

Furthermore, it is import to evaluate the efficiency of VulTriNet. Since multiple datasets were used in this study, to ensure a more accurate evaluation, 800 functions were randomly selected from the combined data as the training set, and 200 functions were selected as the testing set. All approaches were trained for 100 epochs, with their original hyperparameter settings preserved. To mitigate random bias, the experiments were repeated multiple times, and the average results were reported. All training and testing procedures were conducted on the same machine to ensure consistency.

Table 3 shows our results. It can be clearly observed that although VulTriNet is not the fastest approach, it is significantly more efficient than the slowest method. While VulTriNet requires some preprocessing time, it avoids the extensive overhead of complex procedures such as code slicing used in approaches like VulDeePecker. Compared to other models, particularly VulCNN, VulTriNet adopts a more accurate classification architecture, which inevitably introduces some efficiency loss. However, considering that our focus is on static vulnerability detection, the training cost of VulTriNet remains within an acceptable range. Similarly, we present the testing time of VulTriNet in Fig. 15. As shown in the figure, a considerable portion of the total time is spent on channel generation and embedding. However, the actual vulnerability detection process is performed efficiently, which aligns well with our design objective of achieving accurate and timely detection.

6.2.3. Limitation

Despite the encouraging results achieved by our approach, several limitations remain. First, as mentioned earlier, our study focuses on function-level vulnerability detection, which limits its effectiveness

Table 3

Comparison of training time for different approaches.

Approach	Training time (s)
Devign	856.08
VulDeePecker	1944.17
LineVul	1039.80
GraphCodeBERT	1065.16
VulCNN	1131.06
VulTriNet	1361.48

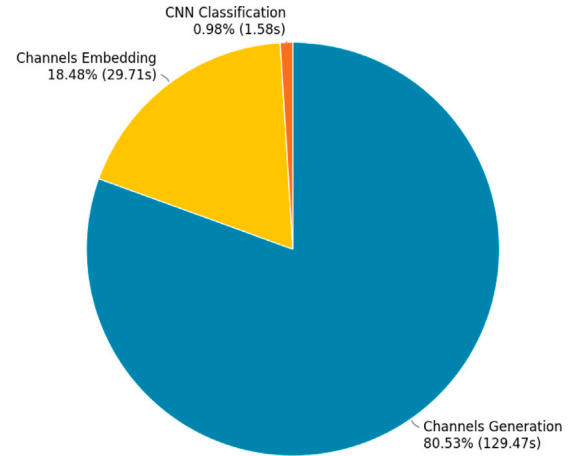


Fig. 15. The total time of VulTriNet on Test(seconds).

in identifying vulnerabilities that span multiple functions. Addressing how to represent and learn from multi-function vulnerabilities will be an important direction for our future work. Second, while we adopt the CodeBERT pre-trained model in this study, there are now more advanced pre-trained models available, such as UniXcoder [47]. Incorporating these models may further improve detection performance and reduce processing time, which we aim to explore in future research. Finally, the embedding dimension in our model is currently limited to 128, which, as shown in this paper and prior studies, is sufficient for effective learning. Nevertheless, investigating higher-dimensional embeddings for enhanced performance or lower-dimensional embeddings for increased efficiency presents another potential avenue for optimization.

7. Related work

In this paper, our work incorporates several advanced techniques for vulnerability detection. In this section, we will discuss the related works in this field.

7.1. Traditional vulnerability detection

In the early stages of vulnerability detection, practitioners relied on expert knowledge to manually construct rules for system identification. The most typical static rule-based analyzers include Rough Audit Tool for Security [48], Checkmarx [49], and FlawFinder [50]. However, since the rules are primarily derived from experienced security experts and each rule describes only a specific vulnerability scenario, this approach is only effective for certain types of vulnerabilities. As a result, it often suffers from high false positive rates (FPR) and false negative rates (FNR) in vulnerability detection.

With the rise of machine learning, manual human-driven methods have gradually been replaced, and many studies employ ML-based approaches to enhance vulnerability detection accuracy and efficiency. In [7,51], source code is treated as text, where statements and tokens

Table 4
Research on some related work.

Approach	Based on text	Based on graph	Embedding vector	Architecture
Automated Detection [12]	✓		Binary Encoding	CNN
Vuldeepecker [11]	✓		Code Gadget, Word2vec	BLSTM
SySeVr [14]	✓		Semantics-based Vulnerability Candidates, Word2vec	BLSTM
μ VulDeePecker [9]	✓		Code Gadget with Control Dependence, Word2vec	Fusion BLSTM
Devign [15]		✓	CPG Graph Embedding	GNN, conv
DeepWukong [54]		✓	PDG, Slicing, Doc2vec	GNN
Reveal [16]		✓	CPG embedding, GGNN embedding	MLP and Softmax
VulCNN [25]		✓	PDG graph to image, Sent2vec	CNN
VulGAI [37]		✓	PDG graph to image, Sent2vec	Efficient CNN
Graph-BiLSTM [55]		✓	CPG graph to sequence, Sent2Vec	BiLSTM
VulBERTa [56]	✓		BPE and custom pre-defined tokens	Transformer, MLP and CNN
LineVul [23]	✓		CodeBERT to Generate Vector	Linear Layer

in the source code are analogous to words in a text. These studies use this textual representation as data features and apply well-known machine learning algorithms, such as Naïve Bayes and Random Forest, to construct vulnerability prediction models for software source codes. In [27], the structural information of source code is utilized by extracting the program's Abstract Syntax Tree (AST) and embedding it into vectors for learning, enabling vulnerability detection. Yamaguchi et al. [52] proposed a method that extends the Code Property Graph to enable automatic inference of taint-style vulnerabilities. Shar et al. [53] proposed a method that enhances the accuracy and scalability of web vulnerability prediction (e.g., SQLI, XSS) by integrating program analysis with machine learning. They combined static slicing and dynamic execution techniques to construct multi-dimensional feature vectors, addressing the limitations of single-analysis approaches. These ML-based methods have provided insights for future research, but their performance is still limited by the constraints of machine learning algorithms.

7.2. Deep learning-based vulnerability detection

With the development of deep learning, more and more research has begun to use deep learning techniques for vulnerability detection, achieving promising results. As shown in Table 4, we provide a concise categorization of existing related work, outlining the general characteristics of each method. We distinguish whether the approach is text-based or graph-based, describe their embedding generation techniques, and summarize their final classification architectures. A more detailed discussion will follow in subsequent sections. Now, many studies have attempted to treat source code as text, using natural language processing methods to convert it into token sequences. Russell et al. [12] is one of the earliest studies to use this technique. Instead of using a neural network as a classifier, it treats source code as text, tokenizes and embeds it, and then trains a neural network to extract features, which are subsequently passed to a random forest classifier for vulnerability detection. Vuldeepecker [11] proposed extracting code gadgets through code slicing, embedding them as feature vectors, and then leveraging Bi-directional Long Short-Term Memory (BiLSTM) for vulnerability detection. However, this method is limited as it only considers vulnerabilities involving library/API function calls. In subsequent work, the authors extended their methodology by proposing SySeVr [14], a systemic framework for vulnerability detection. This framework enhances learning capabilities through the integration of syntactic structures and semantic features from source code. Additionally, they refined the code gadgets approach to more effectively capture both data dependency and control-flow dependency relationships within programs. After that, the authors proposed μ VulDeePecker [9], which builds upon the previous work by incorporating code attention to detect multiple types of vulnerabilities.

In addition to treating source code as a text sequence for processing, other studies have explored converting source code into graphical structures for learning. Devign [15] implemented a Graph Neural

Network (GNN) model to learn the Code Property Graph (CPG) of source code and proposed a novel convolutional component module to effectively extract useful features from rich nodes for classification. DeepWukong [54] converts source code into a Program Dependency Graph (PDG), extracts program slices, and then uses a GNN to detect different types of vulnerabilities. In [16], the authors proposed Reveal, a method that constructs a CPG of the source code, employs a Gated Graph Neural Network (GGNN) to generate graph embeddings from the CPG, and incorporates the Synthetic Minority Oversampling Technique (SMOTE) to address class imbalance caused by scarce vulnerability samples.

In recent years, several studies have integrated graph-based representation methods with image processing techniques to enhance vulnerability detection. VulCNN [25] leveraged the concept of social network centrality to populate the RGB channels of Program Dependence Graph (PDG) images, demonstrating superior performance with CNN-based models over traditional vulnerability detection methods. VulGAI [37] further advances this methodology by adopting a more suitable centrality-based image population strategy and introducing an enhanced CNN-based model specifically designed for vulnerability detection. Furthermore, the study by Ge and Han [55] employed LSTM and innovatively combined the strengths of sequential models and graph mining techniques. They generated various graphs using centrality measures, concatenated their vector representations as sequences, and utilized a BiLSTM for classification. However, their approach still relied on a single type of graph representation and did not incorporate textual semantics, nor did it explore the integration of three distinct code representation methods to populate channels.

7.3. Pre-trained language models for source code

The Transformer [30] model, inspired by NLP's need to understand contextual relationships, replaces sequential processing with self-attention to capture the deep representation of textual statements. Due to its outstanding performance, it has been applied to various text tasks, such as text translation and text generation [57].

After BERT [57] was proposed, many studies have been conducted to improve it and apply it to source code-related fields. CodeBERT [18] enhances BERT by jointly pre-training on code and natural language pairs, enabling cross-modal tasks like code search and documentation generation. CuBERT [58] focuses on syntactic code patterns by training on raw code tokens and AST, optimizing for code classification and clone detection. GraphCodeBERT [24] builds upon CodeBERT, further introducing GNN to model the graphical structure of code. This approach achieved better performance in code understanding and generation tasks.

In recent years, studies have gradually explored the application of pre-trained models to vulnerability detection tasks. VulBERTa [56] is a deep learning method for detecting security vulnerabilities in

source code. It pre-trains a RoBERTa model using real code from open-source C/C++ projects with a customized tokenization pipeline. The authors then use this pre-trained model to train a vulnerability detection classifier. LineVul [23] is a Transformer-based line-level vulnerability prediction method. It leverages the CodeBERT pre-trained language model to generate vector representations of source code. Through large-scale empirical evaluations, it has achieved significant improvements in both function-level and line-level vulnerability prediction.

8. Conclusion

In this paper, we propose VulTriNet, a novel software vulnerability detection method that first undergoes function transformation to generate various function representations, then converts them into usable vectors through embedding vector generation, and finally performs classification using a constructed CNN network. We are the first to utilize AST, PDG, and a BERT pre-trained model as three representation methods to generate images that can be processed by a CNN model. We utilize various advanced embedding vector generation methods to produce usable vectors. Our approach integrates multiple function representations, allowing it to learn more vulnerability-related features compared to other approaches. Experimental results indicate that our approach performs well on two significantly different datasets and demonstrates superiority over several advanced baseline approaches. Furthermore, our ablation and parameter experiments validate that all aspects of our choices are correct.

CRediT authorship contribution statement

Yiyao Yang: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Data curation, Conceptualization. **Youjian Yao:** Writing – original draft, Validation, Data curation. **Xiao Lv:** Writing – review & editing, Validation. **Wen Chen:** Writing – review & editing, Software, Methodology, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Key Research and Development Program of China (020YFB1805405, 2019QY0800), and the Natural Science Foundation of China (62372313, 62002248).

Data availability

Data will be made available on request.

References

- [1] B. Liu, L. Shi, Z. Cai, M. Li, Software vulnerability discovery techniques: A survey, in: 2012 Fourth International Conference on Multimedia Information Networking and Security, IEEE, 2012, pp. 152–156.
- [2] Synopsys, Open source security and risk analysis report. 2023., 2023, <https://www.synopsys.com/zh-cn/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html>.
- [3] J. Newsome, D.X. Song, Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software, in: NDSS, vol. 5, Citeseer, 2005, pp. 3–4.
- [4] B. Johnson, Y. Song, E. Murphy-Hill, R. Bowdidge, Why don't software developers use static analysis tools to find bugs? in: 2013 35th International Conference on Software Engineering, ICSE, IEEE, 2013, pp. 672–681.
- [5] S.M. Ghaffarian, H.R. Shahriari, Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey, ACM Comput. Surv. 50 (4) (2017) 1–36.
- [6] W.U. Ahmad, S. Chakraborty, B. Ray, K.W. Chang, Unified pre-training for program understanding and generation, 2021, arXiv preprint arXiv:2103.06333.
- [7] R. Scandariato, J. Walden, A. Hovsepian, W. Joosen, Predicting vulnerable software components via text mining, IEEE Trans. Softw. Eng. 40 (10) (2014) 993–1006.
- [8] S. Neuhaus, T. Zimmermann, C. Holler, A. Zeller, Predicting vulnerable software components, in: Proceedings of the 14th ACM Conference on Computer and Communications Security, 2007, pp. 529–540.
- [9] D. Zou, S. Wang, S. Xu, Z. Li, H. Jin, μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection, IEEE Trans. Dependable Secur. Comput. 18 (5) (2019) 2224–2236.
- [10] S. Cao, X. Sun, L. Bo, Y. Wei, B. Li, Bgcn4vd: Constructing bidirectional graph neural-network for vulnerability detection, Inf. Softw. Technol. 136 (2021) 106576.
- [11] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, Vuldeepecker: A deep learning-based system for vulnerability detection, 2018, arXiv preprint arXiv:1801.01681.
- [12] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, M. McConley, Automated vulnerability detection in source code using deep representation learning, in: 2018 17th IEEE International Conference on Machine Learning and Applications, ICMLA, IEEE, 2018, pp. 757–762.
- [13] X. Duan, J. Wu, S. Ji, Z. Rui, T. Luo, M. Yang, Y. Wu, VulSniper: Focus your attention to shoot fine-grained vulnerabilities, in: IJCAI, 2019, pp. 4665–4671.
- [14] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, Sysevr: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secur. Comput. 19 (4) (2021) 2244–2258.
- [15] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, Adv. Neural Inf. Process. Syst. 32 (2019).
- [16] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9) (2021) 3280–3296.
- [17] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Adv. Neural Inf. Process. Syst. 25 (2012).
- [18] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint arXiv:2002.08155.
- [19] S. Woo, J. Park, J.Y. Lee, I.S. Kwon, Charn: Convolutional block attention module, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 3–19.
- [20] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, Q. Hu, ECA-net: Efficient channel attention for deep convolutional neural networks, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 11534–11542.
- [21] X. Zhu, D. Cheng, Z. Zhang, S. Lin, J. Dai, An empirical study of spatial attention mechanisms in deep networks, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 6688–6697.
- [22] Software Assurance Reference Dataset (SARD), 2024. <https://samate.nist.gov/SRD/index.php>.
- [23] M. Fu, C. Tantiathamthavorn, Linevul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 608–620.
- [24] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, et al., Graphcodebert: Pre-training code representations with data flow, 2020, arXiv preprint arXiv:2009.08366.
- [25] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, H. Jin, Vulcnn: An image-inspired scalable vulnerability detection system, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2365–2376.
- [26] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: 2014 IEEE Symposium on Security and Privacy, IEEE, 2014, pp. 590–604.
- [27] F. Yamaguchi, M. Lottmann, K. Rieck, Generalized vulnerability extrapolation using abstract syntax trees, in: Proceedings of the 28th Annual Computer Security Applications Conference, 2012, pp. 359–368.
- [28] T. Mikolov, K. Chen, G. Corrado, J. Dean, Efficient estimation of word representations in vector space, 2013, arXiv preprint arXiv:1301.3781.
- [29] M. Pagliardini, P. Gupta, M. Jaggi, Unsupervised learning of sentence embeddings using compositional n-gram features, 2017, arXiv preprint arXiv:1703.02507.
- [30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).
- [31] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: A robustly optimized bert pretraining approach, 2019, arXiv preprint arXiv:1907.11692.
- [32] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, 2014, arXiv preprint arXiv:1409.0473.
- [33] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 7132–7141.
- [34] D. Zou, Y. Hu, W. Li, Y. Wu, H. Zhao, H. Jin, mvulpre: A multi-granularity vulnerability detection system with interpretations, IEEE Trans. Dependable Secur. Comput. (2022).

- [35] L. Moonen, Generating robust parsers using island grammars, in: *Proceedings Eighth Working Conference on Reverse Engineering*, IEEE, 2001, pp. 13–22.
- [36] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, POSTER: Vulnerability discovery with function representation learning from unlabeled projects, in: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 2539–2541.
- [37] C. Zhang, Y. Xin, VulGAI: vulnerability detection based on graphs and images, *Comput. Secur.* 135 (2023) 103501.
- [38] A. Grover, J. Leskovec, node2vec: Scalable feature learning for networks, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [39] S. Zhou, U. Alon, S. Agarwal, G. Neubig, Codebertscore: Evaluating code generation with pretrained models of code, 2023, arXiv preprint [arXiv:2302.05527](https://arxiv.org/abs/2302.05527).
- [40] J. Su, J. Cao, W. Liu, Y. Ou, Whitening sentence representations for better semantics and faster retrieval, 2021, arXiv preprint [arXiv:2103.15316](https://arxiv.org/abs/2103.15316).
- [41] G.H. Golub, C. Reinsch, Singular value decomposition and least squares solutions, in: *Handbook for Automatic Computation: Volume II: Linear Algebra*, Springer, 1971, pp. 134–151.
- [42] H. Abdi, L.J. Williams, Principal component analysis, *Wiley Interdiscip. Rev.: Comput. Stat.* 2 (4) (2010) 433–459.
- [43] Y. Kim, Convolutional neural networks for sentence classification, 2014, URL: <https://arxiv.org/abs/1408.5882>, [arXiv:1408.5882](https://arxiv.org/abs/1408.5882).
- [44] National Institute of Standards and Technology, NIST, 2021, <https://www.nist.gov/>.
- [45] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
- [46] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [47] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, J. Yin, Unixcoder: Unified cross-modal pre-training for code representation, 2022, arXiv preprint [arXiv:2203.03850](https://arxiv.org/abs/2203.03850).
- [48] Rough Audit Tool for Security, Rough audit tool for security, 2021, [Online]. URL: <https://code.google.com/archive/p/roughauditing-tool-for-security/>.
- [49] Checkmarx, Checkmarx, 2021, [Online]. URL: <https://www.checkmarx.com/>.
- [50] FlawFinder, FlawFinde, 2021, [Online]. URL: <http://www.dwheeler.com/flawfinde/r>.
- [51] A. Hovsepian, R. Scandariato, W. Joosen, J. Walden, Software vulnerability prediction using text analysis techniques, in: *Proceedings of the 4th International Workshop on Security Measurements and Metrics*, 2012, pp. 7–10.
- [52] F. Yamaguchi, A. Maier, H. Gascon, K. Rieck, Automatic inference of search patterns for taint-style vulnerabilities, in: *2015 IEEE Symposium on Security and Privacy*, IEEE, 2015, pp. 797–812.
- [53] L.K. Shar, L.C. Briand, H.B.K. Tan, Web application vulnerability prediction using hybrid program analysis and machine learning, *IEEE Trans. Dependable Secur. Comput.* 12 (6) (2014) 688–707.
- [54] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, Deepwukong: Statically detecting software vulnerabilities using deep graph neural network, *ACM Trans. Softw. Eng. Methodol. (TOSEM)* 30 (3) (2021) 1–33.
- [55] K. Ge, Q.B. Han, Hidden code vulnerability detection: A study of the graph-BiLSTM algorithm, *Inf. Softw. Technol.* 175 (2024) 107544.
- [56] H. Hanif, S. Maffei, Vulberta: Simplified source code pre-training for vulnerability detection, in: *2022 International Joint Conference on Neural Networks, IJCNN*, IEEE, 2022, pp. 1–8.
- [57] M.V. Koroteev, BERT: a review of applications in natural language processing and understanding, 2021, arXiv preprint [arXiv:2103.11943](https://arxiv.org/abs/2103.11943).
- [58] A. Kanade, P. Maniatis, G. Balakrishnan, K. Shi, Learning and evaluating contextual embedding of source code, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 5110–5121.